ARCX6 — PARC 2026 Engineers League

Robot & simulation runbook · team ARCX6 · 7 November 2026

Exact commands, launch settings, robot facts, and the errors met during testing with their fixes. Workspace on the laptop: `~/ARCX6FINALREPO/FINAL-ARCX6-PARC/parc_ws`; on the robot: `~/teams/ARCX6`.

Main solution: `explore_solution` — no mapping before the run. Live SLAM on the robot, AprilTag detection with apriltag_ros, drives to every tag it reads, reports only the tag whose turn it is.

1. Run on the real robot

Robot: sitoe@192.168.1.3 (Jetson Orin Nano, aarch64). Same network as the laptop. If the IP changes, replace 192.168.1.3 everywhere below.

1.1 Laptop: clean the robot, zip, copy

```
cd ~/ARCX6FINALREPO/FINAL-ARCX6-PARC/parc_ws
ssh sitoe@192.168.1.3 'rm -f ~/ARCX6.zip && rm -rf ~/teams/ARCX6'
rm -f ARCX6.zip && zip -qr ARCX6.zip src/task_solution src/parcbot_docking \
    src/explore_solution src/tag_tracker src/live_solution src/arcx6_monitor \
    -x '*/__pycache__/*' '*.posegraph' '*.data'
scp ARCX6.zip sitoe@192.168.1.3:~/
```

Only our packages go in the zip (~140 KB): the organisers' parc_* packages are already on the robot. The ssh line deletes the old zip and the whole team folder (src, build, install, log).

1.2 Robot: unzip, build, source

```
mkdir -p ~/teams/ARCX6 && cd ~/teams/ARCX6 && unzip -q ~/ARCX6.zip \
    && colcon build --symlink-install && source install/setup.bash
```

1.3 Before every run

Stop teleop, any separate SLAM and any old solution terminal (Ctrl+C). Two programs driving the wheels or publishing /map at once will fight. Put the robot in its start corner, facing into the arena.

1.4 Run (three terminals on the robot)

```
# Terminal A - the solution
cd ~/teams/ARCX6 && source install/setup.bash
ros2 launch explore_solution explore_solution.launch.py robot:=true

# Terminal B - start the clock (only after the judge ARMs the run on the dashboard)
cd ~/teams/ARCX6 && source install/setup.bash && ros2 run task_solution start_run

# Terminal C - optional live log: tags in view / known / reported, referee, safety, Nav2
cd ~/teams/ARCX6 && source install/setup.bash && ros2 run arcx6_monitor run_monitor
```

Terminal A is ready when it prints `Own velocity commands: cmd_vel_smoothed (TwistStamped, via the collision monitor)`. Its first lines say which camera reads tags (`tags from /camera2/camera2/color/image_raw`) and which Nav2 file is used (`nav2_params_robot.yaml`). The monitor also saves ~/arcx6_logs/monitor_*.log.

1.5 Quick checks

```
ros2 topic list | grep camera2                                        # top camera topics exist
ros2 topic echo /camera2/camera2/color/detections --field detections # tags in view (hold one up)
ros2 topic info /map -v | grep -E "Publisher count|Node name"         # exactly 1: slam_toolbox
ros2 topic info /botwheel_explorer/cmd_vel -v | grep -E "Node name|Endpoint type"  # collision_monitor only
ros2 topic echo /collision_monitor_state --once                       # why it is not moving
ros2 param get /collision_monitor enable_stamped_cmd_vel              # True
```

Useful launch variants: `opening:=none` (no start moves), `opening:="forward 1.3, right 90, forward 1.0"` (other corner), `max_speed:=0.2`, `detector:=opencv`.

1.6 What robot:=true sets

Setting	Value	Why
use_sim_time	false	No simulator clock on the robot

Setting	Value	Why
base_frame	base_link	Robot body frame (the sim uses base_footprint)
localization	slam	Live SLAM: no map, no arena size assumed
Nav2 parameters	config/nav2_params_robot.yaml	Sim uses nav2_params_sim.yaml (same tuning)
max_speed	0.25 m/s	Cautious first runs (robot max 0.5); from the robot params file
wheel commands	/botwheel_explorer/cmd_vel, TwistStamped, frame base_link	Written only by the collision monitor (controller, smoother, monitor all stamped). No relay.
tag camera	camera2 (top, forward)	camera1 is the downward one on the robot (sim: camera1 is the top)
opening	forward / left 90 / forward	Drives out of the start corner before the search (ROBOT_OPENING in the launch file)
depth cameras	camera1, camera2	The robot has both (obstacles)

Every command (Nav2 and the explorer's own moves) passes our collision monitor (lidar + both depth cameras) before the wheels. **Keep someone ready to stop the robot.**

2. How explore_solution works

Step	Behaviour
Before the start	Waits for RUNNING: no movement, no false start.
Start	Spins in place; stops spinning as soon as any tag is read.
Needed tag read (any distance)	Drives there; camera line-up; photo; TagVisit; holds 3.2 s; backs out.
Other tag read	Drives to ~1.3 m, records its position (NOT reported), then scans from there.
No tag in view	Next sweep point, then a scan spin. A tag read while driving interrupts the drive.
Line-up "good enough"	Takes the picture as soon as: camera 0.49-0.61 m from the tag (robot centre ~0.44-0.56 m), within 8 deg of square, within 8 cm of the tag's centre line. Rule: 0.5 m, 12 deg.
Tag not where expected	Sweeps in place after 4 s; if never seen, forgets the position and searches again.
After the order	Missed tags get a second try while >= 60 s remain; then Finish.

Tag reading: apriltag_ros (parcbot_docking pipeline, started automatically) — AprilTag 36h11, IDs 1/4/7/9, exact codes (hamming 0), 2 decodes within 2 s. Distance to tags beyond 1 m comes from the depth camera at the tag's centre pixel. In the sim apriltag_ros read 351 sightings where OpenCV read 10.

Processes (CycloneDDS allows ~10 per host by default): SLAM, Nav2 container (all servers), AprilTag container, depth obstacles, explorer = 5.

3. Log lines to watch (Terminal A)

Line	Meaning
Own velocity commands: cmd_vel_smoothed (TwistStamped, via the collision monitor)	Explorer ready (robot mode)
Opening moves: ... / Opening done; starting the search.	Start-corner moves; "was blocked" = something in the way
Nav2 is still commanding: own move waits.	Normal: explorer waits ~1 s for Nav2 to stop
Camera: N frames ...; tags decoded: 7: 12x, closest 3.3 m; known: [...]	Every 5 s: detection is working
Tag [7] read while driving; going to it now.	New tag seen; heading there
Looking for tag 9: tag 7 spotted; going to measure it (not reporting it yet)	Measuring a non-target tag
Tag 4: pose is within the rules; taking the picture.	Early, rule-compliant photo
Reported tag 4 (1/4). Hold still for 3 s.	TagVisit sent

Line	Meaning
Goal (x, y): robot not moving; skipping.	Unreachable spot; moves on (normal now and then)
Tag N: not found at its estimated position; forgetting it and searching again.	Bad far-away estimate dropped

Tell-tale problems: "tags decoded: none" with a tag clearly in view; the same "skipping" over and over with the robot still; "not visible in the camera; not reporting it".

4. explore_solution launch options

Argument	Default	Use
robot	false	true on the real robot (sets everything in 1.5)
localization	arena	arena = AMCL on the rule-book walls (7 x 6 m, start 1 m from back wall, centred); slam = live map
arena_back / arena_front / arena_left / arena_right	1.0 / 5.0 / 3.5 / 3.5	Arena walls from the start pose (arena mode)
detector	apriltag	apriltag (apriltag_ros) or opencv (fallback)
tag_camera	auto	Camera that reads tags: auto = camera1 (sim), camera2 (robot:=true)
camera_namespace / depth_topic	(from tag_camera)	Override the colour namespace / depth image
opening	auto	Start moves, e.g. "forward 1.3, left 90, forward 1.0"; auto = robot preset, none = off
depth_cameras	camera1,camera2	Only list cameras that exist (a missing one freezes the collision monitor)
max_speed	(from params file)	Override the Nav2 top speed, m/s (files: sim 0.35, robot 0.25)
cmd_vel_topic	/cmd_vel	Sim only: collision monitor output (Twist)
use_composition	true	All Nav2 servers in one process (fewer DDS participants)
use_sim_time / base_frame	true / base_footprint	Simulation values

Live tuning while running, e.g. `ros2 param set /tag_visitor accept_angle_deg 10.0`

5. Run in the simulator (laptop)

Every new terminal: `cd ~/ARCX6FINALREPO/FINAL-ARCX6-PARC/parc_ws && source install/setup.bash`

```
# Terminal 1 - arena on the NVIDIA GPU (prefix applies to this command only), no Gazebo window, no RViz
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia \
__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json \
ros2 launch parc_robot_bringup phase2_hard.launch.py seed:=5 gui:=false rviz:=false

# Terminal 2 - the solution (default: AMCL on the arena walls; add localization:=slam to test live SLAM)
ros2 launch explore_solution explore_solution.launch.py

# Terminal 3 - referee (dashboard http://localhost:8080/judge?token=parc2026)
ros2 launch parc_referee referee.launch.py odom_topic:=/botwheel_explorer/odom

# Terminal 4 - optional RViz, WITHOUT the GPU prefix (RViz fails on NVIDIA offload)
rviz2 -d install/parc_robot_bringup/share/parc_robot_bringup/rviz/task.rviz \
      --ros-args -p use_sim_time:=true

# Arm on the dashboard, then:
ros2 run task_solution start_run
```

Always restart the simulator together with the solution: the solution assumes the robot is at the start pose when it launches. In RViz set the Map display Durability to *Transient Local*.

GPU: the laptop stays in `prime-select on-demand`. Never put the NVIDIA variables in ~/.bashrc, ~/.profile or ~/.config/environment.d — that broke the desktop login. Check with `nvidia-smi` (gz sim should use a few hundred MB).

6. Robot facts (from the robot itself)

Item	Value
Login / workspace	sitoe@192.168.1.3 · ~/teams/ARCX6 (src/, install/)
Body frame	base_link (odom -> base_link exists). No base_footprint.
Odometry	/botwheel_explorer/odom (frame odom, child base_link)
Wheels	/botwheel_explorer/cmd_vel, geometry_msgs/TwistStamped, frame_id base_link
Organisers' teleop	ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/botwheel_explorer/cmd_vel -p stamped:=true -p frame_id:=base_link
Cameras	camera2 = top, forward-looking (tags); camera1 = tilted down. Topics /cameraN/cameraN/color/image_raw (+camera_info, depth/image_rect_raw). In the sim it is the other way round.
Start	In a corner of the arena, walls close behind and to one side
Lidar	/scan, /scan_filtered, /scan_merged
DDS	CycloneDDS, default limits (~10 participants per host). FastRTPS export and raising the limit are not allowed.
Referee	systemd service parc-referee on the robot; arm on the dashboard before start_run
Robot software	The robot runs its own programs (safety chain: /safe_vel, /auto_vel, /stamped_vel, ...). Other teams' leftovers may also run: check ros2 node list.

7. Errors met and their fixes

Error / symptom	Cause	Fix
slam_toolbox: Failed to compute odom pose	Sim time on the robot; base_footprint does not exist	use_sim_time:=false, base_frame:=base_link (robot:=true)
Failed to find a free participant index for domain 0	Too many ROS processes for CycloneDDS	Nav2 composed into one process (use_composition, default)
RTPS_READER_HISTORY Error ...	Mixing FastRTPS and CycloneDDS terminals	Do not export RMW_IMPLEMENTATION

Error / symptom	Cause	Fix
AMCL: "Received a 123 X 117 map" every 5 s	Separate SLAM running next to AMCL	One localizer only: robot:=true (live SLAM)
Scan spin: turned 0 deg / robot never moves	Wrong wheel topic/type; or a missing depth camera blocking the collision monitor	robot:=true: Nav2 stamped, collision monitor -> /botwheel_explorer/cmd_vel; depth_cameras lists only real cameras
Spins and moves but never detects a tag	Tags read from camera1, which looks down on the robot	tag_camera auto = camera2 with robot:=true
Referee is IDLE, not ARMED. Not sending	Run not armed yet	Arm on the dashboard, then start_run
Package '...' not found	Terminal not sourced	source install/setup.bash
RViz: Unable to create a suitable GLXContext	RViz started with the NVIDIA prefix	rviz:=false in the sim launch; run rviz2 separately without the prefix
ruby3.2 (Gazebo) crashed	Terminal still had NVIDIA variables while the NVIDIA driver was off	Open fresh terminals
Tag visible but never recognised	OpenCV cannot read small/far tags	detector:=apriltag (default)
Long turning before the picture	Line-up chased 1.7 deg / 2.5 cm	Good-enough exit (8 deg / 8 cm, rule 12 deg)
Robot stuck under a table	Lidar sees table bases only	Depth-camera obstacles; all moves through the collision monitor
Tag position behind a wall	Far sighting range error	Depth range + snap onto arena walls (arena mode)

8. Packages in the workspace

Package	Role	Status
explore_solution	MAIN. No pre-mapping; finds tags during the run	Sim: reported tags in order; robot preset dry-run OK
task_solution	Shared parts (referee client, detector, visit/line-up, depth_obstacles, Nav2 config); pre-mapped solution (needs a saved map)	Sim: 4/4 tags in 148 s with a saved map
parcbot_docking	apriltag_ros pipeline (rectify + detector), tag_manager	Used by explore_solution
live_solution	Experimental: live SLAM + camera-coverage search	SLAM twisted in the sim; not recommended
tag_tracker	Fallback: no Nav2, potential-field driving	Last resort
arcx6_monitor	Read-only run monitor (ros2 run arcx6_monitor run_monitor)	New; not yet used on a live run

9. Not yet verified

• Behaviour on the real floor: SLAM, odometry, wheel response at 0.25 m/s.
• Printed tag size: the code assumes a 12 cm black square (0.15 m tag with white border) — measure it.
• Newest changes (stamped Nav2 chain without relay, camera2 for tags, opening moves): checked in dry runs and the sim, not yet on a full robot run. The opening turn now uses the SLAM map (odometry over-reports turns): watch that it really turns about 90 deg.
• camera2 topic names on the robot: check with ros2 topic list | grep camera2.

10. Git

```
cd ~/ARCX6FINALREPO/FINAL-ARCX6-PARC
git status                    # pydeps/, models/, *.zip, *.posegraph, .vscode/ are ignored
git add -A && git commit -m "explore_solution: stamped Nav2 chain, camera2 tags, opening moves"
git push origin Philip_dev
```